05. Defect Management Standards

Page is about?

Purpose

*To define a **standardized defect management process** across all Agile QA projects in AGS (Ascensus Government Savings), covering:*

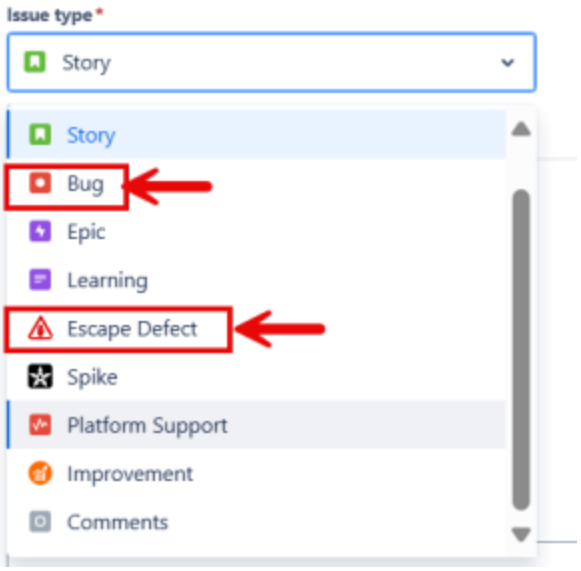
- *Bug & escape defect identification*
- *Logging and lifecycle in **JIRA***
- *Integration with **qTest***
- *Triaging and reporting practices*
- *Email & documentation etiquette*
- *Prevention and continuous improvement*

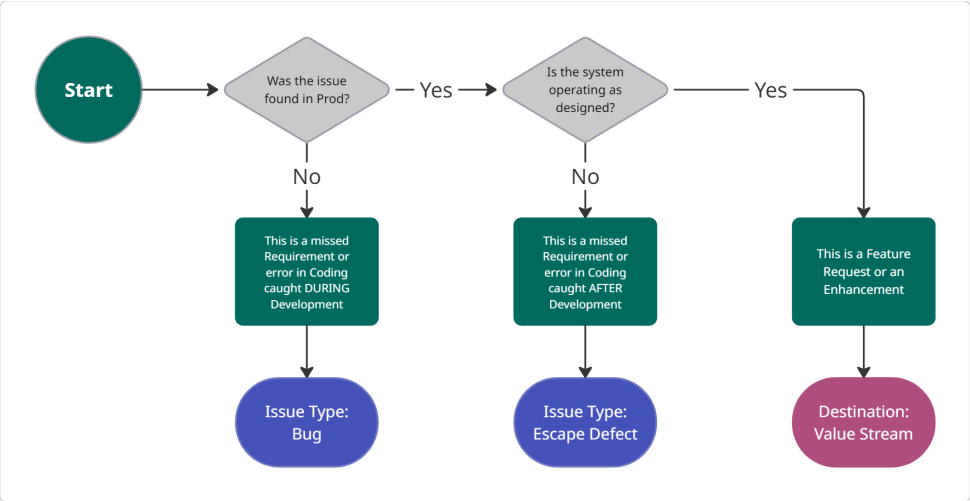
This document ensures defects are handled **consistently, transparently, and effectively** from discovery to closure.

1. Definitions

Term	Description
Bug	A flaw in software that causes incorrect results or behavior. Found during testing or development.
Escape Defect	A defect that **reaches production** undetected through the QA cycle. Indicates process failure.

> *Example: A login crash found in UAT = Bug. Same issue reported by the client after release = Escape Defect.*





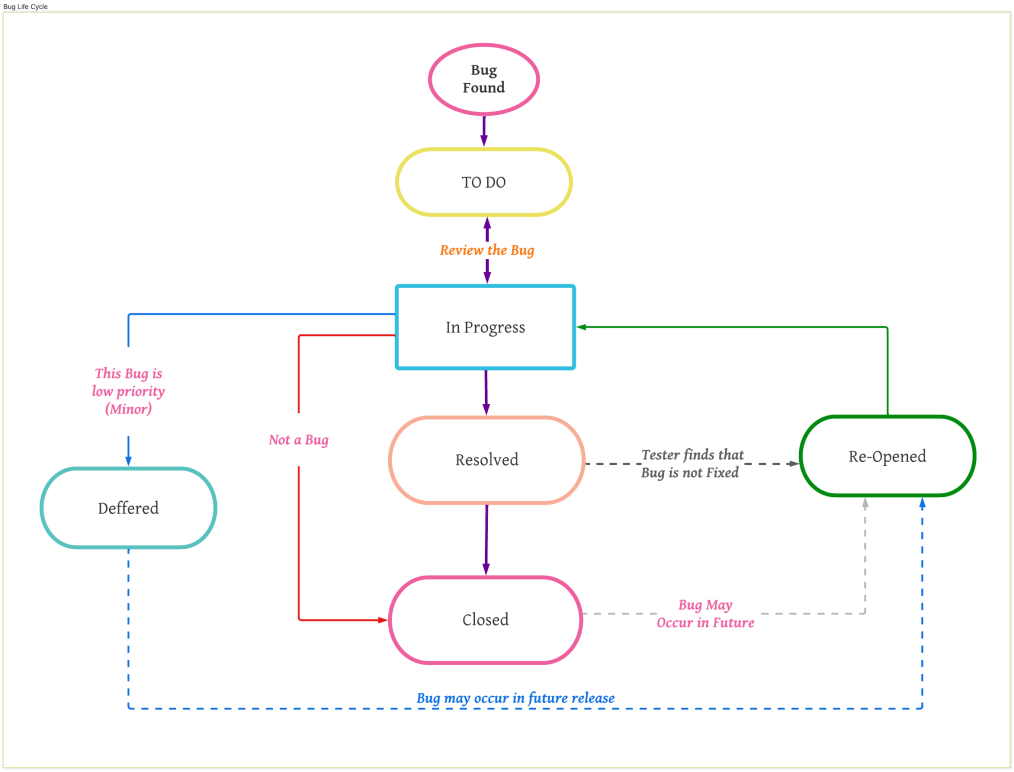
2. Goals of the Defect Management Process (DMP)

- Prevent defects proactively
- Detect early in SDLC
- Reduce rework and defect cost
- Standardize reporting and resolution
- Improve QA process over time

3. Defect Management Lifecycle

Phase	Description
Prevention	Implement code reviews, unit tests, static scans, automation coverage.
Discovery	Log with full evidence (screenshots/logs/steps). Acknowledge by Dev team.
Triage	QA, BA, PO, and Dev review severity, impact, and sprint impact.
Resolution	Fix assigned by priority. Test fix and validate in next QA cycle.
Closure	Once verified by QA, defect is closed. Move to regression set if needed.
Continuous Improvement	Escape defects are analyzed for **root cause** and process gaps.

4. Defect Status Workflow (JIRA Standard)



A simplified standard Jira bug life cycle:

```
Reported  Backlog  To Do  In Progress  Resolved  Closed
               Deferred      Reopened (on fail)
```

> *Use labels like `Escape_Defect`, `Regression_Blocker`, or `Customer_Reported` for better tracking.*

5. Defect Severity vs Priority

Severity	Description	Target Resolution
Blocker	Crashes app or halts testing. No workaround.	Within 1–3 hours
Urgent	Critical flow broken but partial workaround exists.	5–8 hours
Major	Incorrect results or logic failure.	24 hrs
Moderate	Minor visual or intermittent issue.	1–3 days
Minor	Cosmetic or enhancement request.	As agreed by PO

Priority	Description
Critical	Fix immediately. Must go in this release.
High	Fix in the current sprint.
Medium	Fix when possible. Not release blocker.
Low	Optional fix. Defer if needed.

6. qTest Integration with Jira

- All **test cases linked to defects** in Jira.

- Test result outcomes auto-synced.
- Use **given-when-then** style for structured test case design (where applicable).
- Status reported in **qTest dashboards** & tagged in related user stories/bugs.

qTest-Jira Best Practice Link

7. Best Practices to File a Defect

Mandatory Fields for Jira

- Summary (clear & short)
- Description (detailed steps, expected vs actual)
- Priority & Severity
- Affected Environment
- Screenshots or Logs (attach via `Snagit`, `Loom`, or developer logs)
- Labels: `Regression`, `Escape_Defect`, etc.
- Linked test case (if applicable)
- Story/Feature reference

Structure for Description

```
[Issue Summary]
[Steps to Reproduce]
[Expected Behavior]
[Actual Behavior]
[Environment]
[Attachments]
```

8. Sample Email to Report a Defect

> ***Subject***: *[Bug Report] Critical Issue Found – [Short Title] – [Environment]*

Sample Email to Report a Defect

```
Hi [Scrum Team / Developer / BA / PO],

Please find the details of the reported defect below:

 **Summary**: [Short summary]
 **Module**: [e.g., Employee Registration]
 **Severity**: Blocker / Major / Moderate
 **Priority**: Critical / High
 **Environment**: QA / UAT / Production
 **Steps to Reproduce**:
    1. Step 1...
    2. Step 2...
    3. Observe issue.
 **Expected Result**: [What should happen]
 **Actual Result**: [What happens instead]
 **Evidence**: Attached screenshots/video/log
 **JIRA ID**: [JIRA-1234]

Kindly review this at the earliest and advise on next steps.

Thanks,
[Your Name]
[Project Name – QA Team]
[Contact details]
```

9. Folder Structure for Defects & Test Evidence

```
/defects
  /2025
    /Sprint-18
      - JIRA-4321_CriticalLoginIssue.png
      - JIRA-4321_ReproSteps.docx
  /qTestReports
    - LoginModule_Sprint18.qTestExport.xlsx
```

10. Reporting & Metrics

- Jira dashboards track:
    - Defect count per sprint
    - Escape defects
    - Aging defects
    - Severity/Priority trends
- Export defect reports weekly & review in Scrum-of-Scrum meetings.
- Retro: Review any closed defects that were customer-reported or missed in regression.

11. Common Tools

Tool	Use
JIRA	Defect tracking
qTest	Test case linkage & execution
Snagit / Loom/Clipper tool/One note clipper	Capturing screenshots/video
Confluence	Documenting known issues
Teams/Email	Quick triage

12. Root Cause Analysis (RCA) for Escapes

For every **escape defect**, RCA is **mandatory** in sprint retro.

RCA Template Fields:

- Defect Summary
- Test Missed Reason (Gap in test case? No coverage?)
- Prevention Strategy
- Action Owner

13. Continuous Improvement

Post-closure analysis includes:

- Update regression suite.
- Add/modify test cases.
- Update automation for coverage.
- Feedback loop to BA/PO/dev on missed validation steps.

OA Automation Jira Board